

lua-xlsx - Documentation française

Julien Freyermuth

Table des matières

1	lua-xlsx - Documentation française	2
1.1	Table des matières	2
1.2	Architecture et intégration Babet	2
1.3	Conventions	2
1.4	Écriture XLSX	3
1.4.1	xlsx.new([opts]) -> workbook	3
1.4.2	workbook:add_sheet([name]) -> sheet	3
1.4.3	sheet:write(row, col, value [, style]) -> sheet	3
1.4.4	sheet:append_row(values [, style]) -> sheet	4
1.4.5	sheet:write_rows(matrix [, style]) -> sheet	4
1.5	Styles, mise en page et formules	4
1.5.1	xlsx.VERSION	4
1.5.2	xlsx.style([opts]) -> style	4
1.5.3	sheet:set_style(row, col, style) -> sheet	5
1.5.4	sheet:set_column_width(col, width) -> sheet	6
1.5.5	sheet:set_row_height(row, height) -> sheet	6
1.5.6	sheet:freeze_panes(rows, cols) -> sheet	6
1.5.7	sheet:set_auto_filter([range]) -> sheet	6
1.5.8	xlsx.formula(expression) -> formula	6
1.5.9	Exemple complet de rapport présenté	7
1.5.10	workbook:save(path [, opts]) -> true, nil nil, err	7
1.5.11	xlsx.write_rows(path, matrix [, opts])	8
1.6	Dates 1900 et 1904	8
1.6.1	xlsx.date(year, month, day)	8
1.6.2	xlsx.datetime(year, month, day [, hour, minute, second])	8
1.6.3	Utilitaires	9
1.7	Lecture XLSX	9
1.7.1	xlsx.open(path [, opts]) -> workbook, nil nil, err	9
1.7.2	workbook:date_system() -> "1900" "1904"	9
1.7.3	workbook:sheet_names() -> { string, ... }	9
1.7.4	workbook:sheet(which) -> sheet nil [, err]	9
1.7.5	sheet:read(row, col)	9
1.7.6	sheet:dims()	10
1.7.7	sheet:rows()	10
1.8	Limites et validation ZIP	10
1.9	DataFrame	11
1.9.1	Construction	11
1.9.2	Introspection	11
1.9.3	Transformations non destructives	11
1.9.4	Groupement et agrégations	12
1.9.5	Sorties	12
1.10	Contrat d'erreur	13
1.11	Tests et interopérabilité	13
1.12	Génération des PDF	14
1.13	Limites connues	14

1 lua-xlsx - Documentation française

Référence des modules `xlsx` et `dataframe`. La cible principale est Babet avec Lua 5.5, mais le cœur reste compatible avec Lua standard 5.3+.

Documentation de lua-xlsx 1.1.0.

1.1 Table des matières

- [Architecture et intégration Babet](#)
- [Conventions](#)
- [Écriture XLSX](#)
- [Styles, mise en page et formules](#)
- [Dates 1900 et 1904](#)
- [Lecture XLSX](#)
- [Limites et validation ZIP](#)
- [DataFrame](#)
- [Contrat d'erreur](#)
- [Tests et interopérabilité](#)
- [Génération des PDF](#)
- [Limites connues](#)

1.2 Architecture et intégration Babet

`xlsx.lua` et `dataframe.lua` n'exigent aucun module externe. Lorsque la globale `babet` existe, `xlsx.lua` utilise automatiquement les fonctions suivantes :

Fonction Babet	Usage dans lua-xlsx
<code>babet.writeFileAtomic</code>	publication atomique et durable d'un XLSX
<code>babet.archive.test</code>	validation complète du ZIP avant lecture
<code>babet.fileSize</code>	contrôle de taille avant chargement en mémoire
<code>babet.crc32</code>	calcul natif du CRC-32 lors de l'écriture et de la lecture

L'absence d'une fonction précise ne bloque pas la bibliothèque. Le chemin Lua pur reste disponible pour cette opération.

Pour désactiver volontairement l'intégration Babet lors d'un appel :

```
assert(wb:save("rapport.xlsx", { use_babet = false })))
```

```
local read, err = xlsx.open("rapport.xlsx", {  
    use_babet = false,  
})  
assert(read, err)
```

Cette option est utile pour tester le chemin de repli. En production sous Babet, il est généralement préférable de conserver la valeur par défaut `true`.

1.3 Conventions

- `sheet:write(row, col, value [, style])` et `sheet:read(row, col)` utilisent des indices **0-based**. A1 correspond à (0, 0).
- `sheet:rows()` produit des lignes Lua **1-based**. `row[1]` correspond à la colonne A et `row.n` indique la largeur.
- Une cellule vide vaut `nil`. Le booléen `false` reste distinct de `nil`.
- Les dates lues sont des chaînes ISO 8601.
- Les lignes et colonnes respectent les limites XLSX : lignes 0..1048575 et colonnes 0..16383.

- Les chaînes écrites doivent être en UTF-8 valide et ne contenir que des caractères permis par XML 1.0.
- Les transformations DataFrame renvoient de nouveaux records ; elles ne partagent pas les tables de lignes du DataFrame source.

1.4 Écriture XLSX

1.4.1 `xlsx.new([opts]) -> workbook`

Crée un classeur vide.

Option :

Option	Défaut	Valeurs
<code>date_system</code>	"1900"	"1900" ou "1904"

Exemple 1900 :

```
local wb = xlsx.new()
```

Exemple 1904 :

```
local wb = xlsx.new({ date_system = "1904" })
```

Une option inconnue est refusée.

1.4.2 `workbook:add_sheet([name]) -> sheet`

Ajoute une feuille. Le nom par défaut est SheetN.

Règles :

- de 1 à 31 **caractères Unicode**, et non 31 octets ;
- UTF-8 et XML 1.0 valides ;
- pas de `:`, `\`, `/`, `?`, `*`, `[` ou `]` ;
- pas d'apostrophe au début ou à la fin ;
- pas de doublon exact ou de doublon ASCII ne différant que par la casse.

```
local wb = xlsx.new()
local fr = wb:add_sheet("Données")
local unicode = wb:add_sheet(string.rep("é", 20))
```

1.4.3 `sheet:write(row, col, value [, style]) -> sheet`

Types acceptés :

- `string` ;
- entier ou flottant fini ;
- `boolean` ;
- valeur créée par `xlsx.date` ou `xlsx.datetime` ;
- formule créée par `xlsx.formula` ;
- `nil` pour une cellule vide, éventuellement stylée.

```
local sh = xlsx.new():add_sheet("Exemple")
sh:write(0, 0, "Texte")
sh:write(0, 1, 42)
sh:write(0, 2, 3.14)
sh:write(0, 3, false)
sh:write(0, 4, xlsx.date(2026, 7, 18))
sh:write(0, 5, xlsx.formula("SUM(B1:C1)"))
```

NaN, les infinis, les types non pris en charge, les indices négatifs et les indices hors limites Excel lèvent une erreur Lua.

1.4.4 `sheet:append_row(values [, style]) -> sheet`

Ajoute une ligne après la dernière ligne connue.

```
sh:append_row({ "Alice", 30, true })
sh:append_row({ "Bob", 25, false })
```

Les nil terminaux ne peuvent pas être distingués de l'absence d'élément dans un tableau Lua. Pour écrire une cellule éloignée, utiliser `write()`.

1.4.5 `sheet:write_rows(matrix [, style]) -> sheet`

Ajoute une matrice complète :

```
sh:write_rows({
  { "Nom", "Score" },
  { "Alice", 18 },
  { "Bob", 15 },
})
```

1.5 Styles, mise en page et formules

Les fonctions de cette section concernent l'écriture. L'API de lecture renvoie les valeurs des cellules, mais n'expose pas encore les styles, les dimensions, les filtres ni les expressions de formules.

1.5.1 `xlsx.VERSION`

La version du module est disponible sous forme de chaîne :

```
assert(xlsx.VERSION == "1.1.0")
```

1.5.2 `xlsx.style([opts]) -> style`

Crée un style réutilisable et immuable. Un style doit être créé avec `xlsx.style`; une table d'options brute n'est pas acceptée par `write()`.

Option	Type	Valeurs
<code>bold</code>	booléen	police grasse
<code>italic</code>	booléen	police italique
<code>font_color</code>	chaîne	RGB RRGGBB, #RRGGBB ou ARGB AARRGGBB
<code>fill_color</code>	chaîne	fond plein RGB ou ARGB
<code>horizontal</code>	chaîne	left, center, right, justify
<code>vertical</code>	chaîne	top, center, bottom, justify
<code>wrap_text</code>	booléen	retour automatique à la ligne
<code>number_format</code>	chaîne	alias documenté ou code de format Excel

Les couleurs RGB à six chiffres reçoivent automatiquement un canal alpha opaque FF.

Police grasse :

```
local bold = xlsx.style({ bold = true })
sh:write(0, 0, "Important", bold)
```

Italique et couleur de police :

```
local note = xlsx.style({
  italic = true,
  font_color = "7F6000",
})
sh:write(1, 0, "Note", note)
```

Fond plein :

```
local warning = xlsx.style({ fill_color = "FFF2CC" })
sh:write(2, 0, "Attention", warning)
```

Alignement horizontal :

```
local centered = xlsx.style({ horizontal = "center" })
sh:write(3, 0, "Centré", centered)
```

Alignement vertical et retour à la ligne :

```
local wrapped = xlsx.style({
  vertical = "center",
  wrap_text = true,
})
sh:write(4, 0, "Texte long sur plusieurs lignes", wrapped)
```

Formats numériques prédéfinis :

Alias	Code Excel produit
general	aucun format personnalisé
integer	0
decimal	0.00
percent	0.00%
currency_eur	#,##0.00 "€"
currency_usd	\$#,##0.00
date	yyyy-mm-dd
datetime	yyyy-mm-dd hh:mm:ss

Un alias par usage :

```
sh:write(0, 1, 42, xlsx.style({ number_format = "integer" }))
sh:write(1, 1, 3.14159, xlsx.style({ number_format = "decimal" }))
sh:write(2, 1, 0.125, xlsx.style({ number_format = "percent" }))
sh:write(3, 1, 19.90, xlsx.style({ number_format = "currency_eur" }))
```

Un code Excel personnalisé est également accepté :

```
local code = xlsx.style({ number_format = '0.000 "kg"' })
sh:write(4, 1, 12.3456, code)
```

Les valeurs `xlsx.date` et `xlsx.datetime` reçoivent automatiquement leur format date si le style fourni ne définit pas déjà `number_format`. Il est donc possible de combiner gras et date sans perdre le format calendaire :

```
local date_bold = xlsx.style({ bold = true })
sh:write(0, 2, xlsx.date(2026, 7, 18), date_bold)
```

Exemple combiné de style d'en-tête :

```
local header = xlsx.style({
  bold = true,
  font_color = "FFFFFF",
  fill_color = "4472C4",
  horizontal = "center",
  vertical = "center",
  wrap_text = true,
})
sh:append_row({ "Produit", "Prix", "Date" }, header)
```

1.5.3 sheet:set_style(row, col, style) -> sheet

Applique un style après l'écriture, ou à une cellule vide :

```
sh:write(0, 0, "Titre")
sh:set_style(0, 0, xlsx.style({ bold = true }))
```

```
-- cellule vide mais colorée
sh:set_style(1, 0, xlsx.style({ fill_color = "E2F0D9" })))
```

Passer nil retire le style enregistré pour cette cellule :

```
sh:set_style(0, 0, nil)
```

1.5.4 sheet:set_column_width(col, width) -> sheet

col est 0-based. La largeur doit être un nombre fini compris entre 0.1 et 255 unités Excel.

```
sh:set_column_width(0, 24) -- colonne A
sh:set_column_width(1, 14) -- colonne B
```

1.5.5 sheet:set_row_height(row, height) -> sheet

row est 0-based. La hauteur est exprimée en points et doit être comprise entre 0.1 et 409.5.

```
sh:set_row_height(0, 28)
```

1.5.6 sheet:freeze_panes(rows, cols) -> sheet

rows et cols sont le nombre de lignes et de colonnes à conserver visibles depuis le coin supérieur gauche.

Figurer seulement la première ligne :

```
sh:freeze_panes(1, 0) -- première cellule libre : A2
```

Figurer seulement la première colonne :

```
sh:freeze_panes(0, 1) -- première cellule libre : B1
```

Figurer la première ligne et la première colonne :

```
sh:freeze_panes(1, 1) -- première cellule libre : B2
```

freeze_panes(0, 0) supprime le gel.

1.5.7 sheet:set_auto_filter([range]) -> sheet

Sans argument, le filtre couvre la zone utilisée au moment de la sauvegarde :

```
sh:set_auto_filter()
```

Une plage explicite doit être rectangulaire, en notation A1, dans l'ordre coin supérieur gauche vers coin inférieur droit :

```
sh:set_auto_filter("A1:D100")
```

Passer false désactive le filtre :

```
sh:set_auto_filter(false)
```

Cette fonction ajoute les boutons de filtre Excel. Elle ne filtre pas les lignes dans le fichier produit.

1.5.8 xlsx.formula(expression) -> formula

Crée une formule destinée à l'écriture. Le signe = initial est facultatif :

```
sh:write(3, 1, xlsx.formula("SUM(B1:B3)"))
sh:write(3, 2, xlsx.formula("=AVERAGE(C1:C3)"))
```

La formule est stockée dans le classeur, mais lua-xlsx ne possède pas de moteur de calcul. Il n'écrit donc pas de résultat mis en cache. Excel, LibreOffice ou un autre tableur calcule la formule à l'ouverture. Un lecteur utilisant le mode data_only peut voir nil tant que le classeur n'a pas été recalculé et sauvegardé par un tableur.

Une formule peut recevoir un style comme toute autre cellule :

```
local money = xlsx.style({ number_format = "currency_eur" })
sh:write(10, 3, xlsx.formula("SUM(D2:D10)"), money)
```

1.5.9 Exemple complet de rapport présenté

```
local xlsx = require("xlsx")

local wb = xlsx.new()
local sh = wb:add_sheet("Ventes")

local header = xlsx.style({
  bold = true,
  font_color = "FFFFFF",
  fill_color = "4472C4",
  horizontal = "center",
  vertical = "center",
  wrap_text = true,
})
local money = xlsx.style({ number_format = "currency_eur" })

sh:append_row({ "Produit", "Prix", "Quantité", "Total" }, header)
sh:append_row({ "Clavier", 39.90, 2 })
sh:append_row({ "Souris", 24.50, 3 })

sh:write(1, 1, 39.90, money)
sh:write(2, 1, 24.50, money)
sh:write(1, 3, xlsx.formula("B2*C2"), money)
sh:write(2, 3, xlsx.formula("B3*C3"), money)
sh:write(3, 3, xlsx.formula("SUM(D2:D3)"), money)

sh:set_column_width(0, 24)
sh:set_column_width(1, 14)
sh:set_column_width(2, 12)
sh:set_column_width(3, 16)
sh:set_row_height(0, 28)
sh:freeze_panes(1, 1)
sh:set_auto_filter("A1:D3")

assert(wb:save("ventes.xlsx"))
```

1.5.10 workbook:save(path [, opts]) -> true, nil | nil, err

Options :

Option	Défaut	Effet
overwrite	true	remplace un fichier existant
durable	true	demande la synchronisation durable sous Babet
permissions	0644	permissions du fichier sous Babet
use_babet	true	utilise writeFileAtomic si disponible

Écriture simple :

```
local ok, err = wb:save("rapport.xlsx")
assert(ok, err)
```

Refuser un remplacement :

```
local ok, err = wb:save("rapport.xlsx", {
  overwrite = false,
})
```

Cache reconstituable sans attente de durabilité :

```
assert(wb:save("cache.xlsx", {
    durable = false,
}))
```

Permissions privées :

```
assert(wb:save("privé.xlsx", {
    permissions = tonumber("600", 8),
}))
```

Exemple combiné :

```
local ok, err = wb:save("export/rapport.xlsx", {
    overwrite = true,
    durable = true,
    permissions = tonumber("640", 8),
    use_babet = true,
})
assert(ok, err)
```

Sous Babet, l'appel passe par `babet.writeFileAtomic`. Sous Lua standard, le module écrit un fichier temporaire dans le même dossier, contrôle `write`, `flush` et `close`, puis le renomme. Ce repli fournit une publication atomique sur les systèmes POSIX usuels, mais ne reproduit pas toutes les garanties de confinement et de `fsync` de Babet.

1.5.11 `xlsx.write_rows(path, matrix [, opts])`

Options :

- `sheet` : nom de feuille, défaut `Sheet1` ;
- `date_system` : 1900 ou 1904 ;
- `overwrite`, `durable`, `permissions`, `use_babet` : mêmes options que `save()`.

```
assert(xlsx.write_rows("résultat.xlsx", {
    { "x", "y" },
    { 1, 2 },
    { 3, 4 },
}, {
    sheet = "Résultats",
    date_system = "1900",
    overwrite = true,
}))
```

1.6 Dates 1900 et 1904

1.6.1 `xlsx.date(year, month, day)`

```
sh:write(0, 0, xlsx.date(2026, 7, 18))
```

1.6.2 `xlsx.datetime(year, month, day [, hour, minute, second])`

```
sh:write(0, 1, xlsx.datetime(2026, 7, 18, 13, 45, 30))
```

Les constructeurs valident :

- année 1..9999 ;
- mois 1..12 ;
- nombre de jours réel du mois, années bissextiles comprises ;
- heure 0..23, minute et seconde 0..59 ;
- arguments entiers.

1.6.3 Utilitaires

```
local serial1900 = xlsx.date_to_serial(2026, 7, 18)
local serial1904 = xlsx.date_to_serial(2026, 7, 18, 0, 0, 0, true)

print(xlsx.serial_to_iso(serial1900, false))
print(xlsx.serial_to_iso(serial1904, false, true))
```

Le classeur écrit le bon numéro de série selon date_system. À la lecture, <workbookPr date1904="1"/> est détecté automatiquement.

1.7 Lecture XLSX

1.7.1 xlsx.open(path [, opts]) -> workbook, nil | nil, err

L'ouverture suit les étapes suivantes :

1. contrôle de la taille du fichier ;
2. validation par `babet.archive.test()` lorsqu'elle est disponible ;
3. chargement borné en mémoire ;
4. vérification du répertoire central ZIP et des en-têtes locaux ;
5. extraction et contrôle CRC des parties XML utilisées ;
6. parsing du classeur, des chaînes partagées et des styles.

```
local wb, err = xlsx.open("rapport.xlsx")
assert(wb, err)
```

Désactiver uniquement la prévalidation Babet :

```
local wb, err = xlsx.open("rapport.xlsx", {
    validate_archive = false,
})
```

Le parseur Lua conserve néanmoins ses propres vérifications.

1.7.2 workbook:date_system() -> "1900" | "1904"

```
print(wb:date_system())
```

1.7.3 workbook:sheet_names() -> { string, ... }

```
for _, name in ipairs(wb:sheet_names()) do
    print(name)
end
```

1.7.4 workbook:sheet(which) -> sheet | nil [, err]

which est un nom ou un index entier 1-based.

```
local first = wb:sheet(1)
local data = wb:sheet("Données")
```

Une feuille est extraite et parsée lors du premier accès, puis mise en cache. Une corruption propre à cette feuille est donc renvoyée par `sheet()` sous forme `nil, err`.

1.7.5 sheet:read(row, col)

```
local value = sheet:read(0, 0)
```

Valeurs possibles : chaîne, nombre, booléen, chaîne ISO de date, ou `nil`. Les cellules d'erreur OOXML sont actuellement renvoyées sous forme de chaîne.

1.7.6 sheet:dims()

```
local maxrow, maxcol = sheet:dims()
```

Une feuille vide renvoie -1, -1.

1.7.7 sheet:rows()

```
for row in sheet:rows() do
  for col = 1, row.n do
    print(row[col])
  end
end
```

Les lignes vides intermédiaires sont produites afin de préserver les numéros de ligne du tableur.

1.8 Limites et validation ZIP

Options de `xlsx.open` :

Option	Défaut	Plafond accepté
max_file_size	256 Mio	pas de plafond interne supplémentaire
max_entries	10 000	100 000
max_entry_size	64 Mio	8 Gio
max_total_size	512 Mio	64 Gio
max_path_length	4 096 octets	1 Mio
max_total_name_bytes	16 Mio	64 Mio
max_compression_ratio	200	1 000 000 000
use_babet	true	booléen
validate_archive	true	booléen

Un exemple par limite :

```
xlsx.open("a.xlsx", { max_file_size = 32 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_entries = 2000 })
xlsx.open("a.xlsx", { max_entry_size = 16 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_total_size = 128 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_path_length = 1024 })
xlsx.open("a.xlsx", { max_total_name_bytes = 1024 * 1024 })
xlsx.open("a.xlsx", { max_compression_ratio = 100 })
```

Exemple combiné pour un fichier reçu d'un tiers :

```
local wb, err = xlsx.open("import.xlsx", {
  max_file_size = 64 * 1024 * 1024,
  max_entries = 5000,
  max_entry_size = 16 * 1024 * 1024,
  max_total_size = 128 * 1024 * 1024,
  max_path_length = 2048,
  max_total_name_bytes = 2 * 1024 * 1024,
  max_compression_ratio = 100,
  use_babet = true,
  validate_archive = true,
})
assert(wb, err)
```

Le lecteur Lua refuse notamment :

- ZIP multidisque, ZIP64, chiffrement et méthodes autres que STORED/DEFLATE ;
- noms absolus, traversées . . , backslashes, doublons et noms non UTF-8 ;
- en-têtes locaux incohérents, data descriptors incohérents et chevauchements ;

- tailles annoncées incohérentes, CRC incorrects et flux DEFLATE tronqués ;
- sortie décompressée ou rapport de compression au-delà des limites.

1.9 DataFrame

Un DataFrame possède :

- columns : noms ordonnés, chaînes non vides et uniques ;
- rows : records nom -> valeur.

1.9.1 Construction

1.9.1.1 DF.from_rows(matrix [, opts]) Options :

Option	Effet
header	la première ligne fournit les noms
columns	noms explicites si header est faux

```
local d = DF.from_rows({
  { "nom", "âge" },
  { "Alice", 30 },
  { "Bob", 25 },
}, { header = true })
```

Les colonnes vides générées par un header nil deviennent colN. Les doublons de noms sont refusés.

1.9.1.2 DF.from_records(records [, columns])

```
local d = DF.from_records({
  { nom = "Alice", âge = 30 },
  { nom = "Bob", âge = 25 },
}, { "nom", "âge" })
```

Sans columns, les clés textuelles sont déduites et triées.

1.9.1.3 DF.from_sheet(sheet [, opts])

```
local d = DF.from_sheet(assert(wb:sheet("Données")), {
  header = true,
})
```

1.9.2 Introspection

```
print(d:nrow(), d:ncol())
print(table.concat(d:colnames(), ", "))
local ages = d:column("âge")
```

d:iter() renvoie une copie de chaque record :

```
for row in d:iter() do
  row.âge = 0 -- ne modifie pas d
end
```

1.9.3 Transformations non destructives

1.9.3.1 filter

```
local adults = d:filter(function(row)
  return row.âge >= 18
end)
```

Le callback reçoit une copie du record.

1.9.3.2 select

```
local names = d:select("nom")
local same = d:select({ "nom", "âge" })
```

Une même colonne ne peut pas être sélectionnée deux fois.

1.9.3.3 rename

```
local renamed = d:rename({ âge = "age" })
```

Une source inconnue, un nouveau nom vide ou une collision sont refusés.

1.9.3.4 mutate

```
local with_total = d:mutate("double_age", function(row)
  return row.âge * 2
end)
```

Le callback reçoit une copie du record source.

1.9.3.5 sort

```
local asc = d:sort("âge")
local desc = d:sort("âge", { desc = true })
```

Le tri est stable et place toujours les nil à la fin.

1.9.3.6 head et tail

```
local first = d:head(10)
local last = d:tail(10)
```

n doit être un entier positif ou nul. La valeur par défaut est 5.

1.9.4 Groupement et agrégations

```
local grouped = ventes:groupby("ville", "produit"):agg({
  total = { "sum", "montant" },
  moyenne = { "mean", "montant" },
  minimum = { "min", "montant" },
  maximum = { "max", "montant" },
  premier = { "first", "montant" },
  dernier = { "last", "montant" },
  nombre = { "count" },
})
```

Fonctions : sum, mean/avg, min, max, count, first, last.

Les clés de groupe prennent en charge nil, booléens, chaînes binaires et nombres. La sérialisation interne est préfixée par type et longueur ; une chaîne contenant des séparateurs ou des octets NUL ne peut donc plus fusionner deux groupes distincts.

groupby:count() équivaut à :

```
local counts = d:groupby("ville"):agg({ n = { "count" } })
```

1.9.5 Sorties

```
local records = d:to_records()
local rows = d:to_rows()
local rows_without_header = d:to_rows({ header = false })
print(d:tostring(20))
d:show(20)
```

1.10 Contrat d'erreur

Les erreurs d'utilisation lèvent une erreur Lua :

- mauvais type ou mauvaise option ;
- nom de colonne ou feuille invalide ;
- date impossible ;
- indice hors limites ;
- transformation DataFrame incohérente.

Les erreurs de système ou de données renvoient normalement nil, err :

```
local wb, err = xlsx.open("absent.xlsx")
if not wb then
    io.stderr:write(err, "\n")
end
```

workbook:sheet() suit le même principe pour une feuille corrompue ou une partie manquante.

Workbook:save() vérifie les résultats d'écriture, de flush, de fermeture et de publication. Il ne renvoie plus true après une écriture réellement échouée.

1.11 Tests et interopérabilité

./run_tests.sh

Le harnais vérifie notamment :

- écriture et relecture ;
- chaînes Unicode et entités XML ;
- CRC corrompu ;
- systèmes de dates 1900 et 1904 ;
- styles, formats numériques et dates stylées ;
- largeurs, hauteurs, volets figés, filtres et formules ;
- limites Excel et XML 1.0 ;
- appels automatiques à Babet ;
- non-destruction des DataFrames ;
- absence de collision dans groupby ;
- aller-retour réel avec openpyxl.

Babet est recherché dans cet ordre :

1. variable BABET_BIN ;
2. binaire local bin/babet ;
3. commande babet disponible dans le PATH.

Le script crée ensuite un venv Python temporaire, installe openpyxl>=3.1,<4, exécute l'interopérabilité séparément avec Babet et Lua standard lorsqu'ils sont disponibles, puis supprime le venv, le cache pip et les fichiers temporaires. Le nettoyage est assuré aussi lorsqu'un test échoue. Une installation globale d'openpyxl n'est donc ni utilisée ni nécessaire.

Prérequis de cette phase : python3, le module venv, pip dans le venv et un accès au dépôt Python configuré. Sous Debian ou Ubuntu, installer python3-venv si la création du venv échoue.

Utiliser le binaire local :

```
cp /chemin/vers/babet bin/babet
chmod +x bin/babet
./run_tests.sh
```

Forcer les runtimes ou la version d'openpyxl :

```
BABET_BIN=../babet/bin/babet ./run_tests.sh
LUA_BIN=/usr/bin/lua5.5 ./run_tests.sh
```

```
OPENPYXL_SPEC='openpyxl==3.1.5' ./run_tests.sh
```

1.12 Génération des PDF

```
cd doc  
./build_doc.sh
```

Sorties :

- documentation-fr.pdf ;
- documentation-en.pdf.

Prérequis : Pandoc et XeLaTeX ou LuaLaTeX. Le script utilise un répertoire temporaire et ne laisse pas les fichiers auxiliaires LaTeX dans le projet.

1.13 Limites connues

- Écriture ZIP STORED uniquement ; lecture STORED et DEFLATE.
- Pas de ZIP64 dans le parseur Lua interne.
- Pas de cellules fusionnées, graphiques, images, validations de données ni mise en forme conditionnelle.
- Les formules peuvent être écrites, mais lua-xlsx ne les calcule pas et ne lit pas encore leur expression. Seule une éventuelle valeur mise en cache est lue.
- Les styles, dimensions, volets et filtres sont produits en écriture, mais ne sont pas exposés par l'API de lecture.
- Pas de lecture en streaming : le fichier et les XML utiles restent en mémoire.
- Parsing XML spécialisé au sous-ensemble XLSX utilisé, pas parseur XML général.
- Pas de normalisation Unicode ou de case folding Unicode complet pour les noms de feuilles ; la détection insensible à la casse est fiable pour l'ASCII.